

Aula 2 (Módulo 07) — Pull Requests e Issues

1. Pull Request (PR) — pedindo para juntar seu trabalho

Na Aula 1 vimos `git merge` rodando localmente, no seu próprio computador. Um **Pull Request** (PR) é a versão "formal e visível" desse merge, feita através do GitHub: você propõe que os commits de uma branch sejam incorporados a outra (geralmente a `main`), e o PR fica como uma página onde é possível:

- ver o **diff** completo (todas as linhas adicionadas/removidas);
- deixar **comentários** em linhas específicas do código;
- rodar **checks automatizados** (testes, linters — verificações automáticas de qualidade/estilo de código);
- exigir **aprovação** de outra pessoa antes de poder ser mesclado.

```
git push origin minha-branch
# depois, pelo site do GitHub: "Compare & pull request"
```

Por que importa mesmo sozinho: em um projeto pessoal (como este repositório), você pode achar que PR é desnecessário — mas ele cumpre um papel importante mesmo assim: cria um **registro histórico** de "o que mudou e por quê" em um nível mais alto que commits individuais, e é exatamente o fluxo que um recrutador técnico espera ver num portfólio maduro (mostra que você sabe trabalhar como profissionais trabalham em equipe, mesmo em projeto solo).

2. Code Review — o valor de um PR revisado

Quando um PR tem **revisores** (outras pessoas, ou até você mesmo revendo com calma antes de aceitar), eles podem:

- **Aprovar** (`Approve`);
- Pedir **mudanças** (`Request changes`) — comentando exatamente onde e por quê;
- Só **comentar**, sem bloquear (`Comment`).

Por que importa para segurança especificamente: muitos ataques à cadeia de suprimento de software (*supply chain attacks*) começam com código malicioso sendo introduzido sem revisão adequada — um PR bem revisado é uma camada de defesa. Times de segurança também usam PR para revisar mudanças em **regras de detecção** (Módulo 21) ou em infraestrutura crítica antes delas irem para produção.

3. Issues — rastreando tarefas, bugs e ideias

Uma **Issue** é um item rastreável no GitHub — pode ser um bug ("comando X não funciona quando Y"), uma tarefa ("adicionar aula sobre Z"), ou uma pergunta. Diferente de um PR, uma Issue **não** tem código associado diretamente — é só uma discussão/registro.

Boas práticas de Issue:

- **Título claro e específico:** "Aula 3 do Módulo 02 tem link quebrado" é melhor que "erro".
- **Labels** (etiquetas): categorizam o tipo (`bug` , `enhancement` , `documentation` ...) e facilitam filtrar depois.

- **Referenciar no commit/PR:** escrever `Closes #12` na descrição de um PR faz o GitHub fechar automaticamente a Issue #12 quando aquele PR for mesclado — conectando "o que foi pedido" com "o que resolveu".

Por que importa para portfólio: um histórico de Issues bem escritas mostra capacidade de **comunicação técnica clara** — uma habilidade tão valorizada em vagas de SOC/Blue Team quanto habilidade técnica pura (lembra do CLAUDE.md deste repositório: "como documentar", "como comunicar" fazem parte do que você está aprendendo aqui).

4. Fork vs. Branch — dois jeitos de contribuir

- **Branch:** você tem permissão de escrita direta no repositório (é o seu próprio projeto, ou um projeto de equipe onde você é colaborador) — cria uma branch dentro do mesmo repo.
- **Fork:** você **não** tem permissão de escrita (ex.: quer contribuir para um projeto open source de terceiros) — o GitHub cria uma **cópia completa** do repositório na sua conta, você trabalha nela, e depois abre um PR **do seu fork para o repositório original**.

```
# depois de dar fork pelo site do GitHub:  
git clone https://github.com/SEU_USUARIO/nome-do-projeto  
cd nome-do-projeto  
git remote add upstream https://github.com/DONO_ORIGINAL/nome-do-projeto
```

`upstream` é a convenção de nome para o repositório **original**, diferente do seu `origin` (seu próprio fork) — permite trazer atualizações do projeto original para o seu fork depois.

5. Conectando com o fluxo que já usamos neste repositório

```
Branch (Aula 1) → trabalho isolado  
↓  
git push origin branch → sincroniza com o GitHub  
↓  
Pull Request (esta aula) → propõe juntar à main, com diff visível,  
                           espaço para comentário/revisão  
↓  
Merge (aprovado) → código entra na main
```

As Issues, nesse fluxo, existem **antes** de tudo isso — normalmente é uma Issue que descreve o que precisa ser feito, e o PR é o que efetivamente faz.

6. Exercício prático

No GitHub, na página do seu próprio repositório de estudo:

1. Crie uma Issue descrevendo uma pequena melhoria que você gostaria de fazer (ex.: "Adicionar exercício extra na Aula X").

2. Crie uma branch, faça a mudança, e abra um Pull Request referenciando essa Issue (`Closes #N` , onde N é o número da Issue).
3. Revise seu próprio PR como se fosse outra pessoa: o diff está claro? A mensagem de commit explica bem a mudança?
4. Faça o merge.

7. Recapitulando

- Pull Request = pedido formal e visível para juntar uma branch, incluindo diff, comentários e (opcionalmente) checks automatizados e aprovação obrigatória.
- Code review é uma camada de defesa contra código malicioso/erros, inclusive em regras de detecção.
- Issues rastreiam tarefas/bugs/ideias; boas Issues (título claro, labels, referência no PR) demonstram comunicação técnica.
- Fork é usado quando você não tem escrita direta no repositório; branch quando tem.

Próxima aula (fecha o Módulo 07)

README profissional e organização de portfólio — como estruturar e documentar projetos de segurança para que um recrutador consiga navegar e entender rapidamente o que você fez.

Fontes recomendadas

- **GitHub Docs — "About pull requests"**: documentação oficial completa sobre PRs. <https://docs.github.com/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>
- **GitHub Docs — "About issues"**: documentação oficial sobre Issues. <https://docs.github.com/issues/tracking-your-work-with-issues/about-issues>
- **GitHub Docs — "Fork a repo"**: documentação oficial sobre forks. <https://docs.github.com/pull-requests/collaborating-with-pull-requests/working-with-forks/fork-a-repo>